Privilege Escalation — Card Catalog (not a dictionary)

The rule for using this doc: you're not memorizing commands. You're memorizing that a *tool exists* for a *specific question*. The exact flags are always 5 seconds away in a terminal (`--help`, `man`, or GTFOBins/LOLBAS). What you actually need permanently stored in your head is the **left column** — the question. The right column is just where to look it up.

The one meta-question behind everything

Q: I've landed on a box as a low-privileged user. What do I even do?

> A: Find something that runs with more privilege than me, figure out what it *trusts* (a file, a path, a setting, a credential), and check if I can touch that trust. If yes, redirect it or use it directly.

Everything below is just this question, asked in a different corner of the OS.

LINUX

Recon — "what am I working with?"

Q: What OS/kernel am I on, and is it old enough to have a known exploit?

> A: `uname -a`, `cat /etc/issue`, `cat /proc/version` — then check that version against searchsploit or Google.

Q: What am I allowed to do as this user?

> A: `id` (groups), `sudo -l` (root-level commands I'm pre-approved for).

Q: What's currently running on this machine, and who started it?

> A: `ps aux` (all processes), `ps axjf` (as a tree, shows parent→child relationships).

Q: How does the system find commands when I type their name?

> A: The `PATH` environment variable (`echo $PATH`) — an ordered list of directories it searches. This matters later for hijacking.

Q: Where do account and password details live?

> A: `/etc/passwd` (public account metadata) vs. `/etc/shadow` (actual password hashes, root-only).

Q: How do I search the whole filesystem for something specific?

> A: `find` — the tool exists; the exact flags depend on what you're hunting (see below).

Technique catalog — "what's the actual lever?"

Q: Is there a bug in the kernel itself I can exploit?

> A: Check version with `uname -a`, search for a matching public exploit, compile and run it. ⚠️ Last resort — can crash the box. Real risk in production, common in labs.

Q: Am I allowed to run something as root that I can abuse beyond its intended purpose?

> A: `sudo -l` shows what I'm allowed to run → look the binary up on **GTFOBins** for its shell-escape syntax.

Q: Is there a binary that always runs as its owner, regardless of who executes it?

> A: That's a SUID bit. Find them with `find / -perm -4000 2>/dev/null` → check GTFOBins for abuse if the binary is unusual (not a standard system tool).

Q: Is there a binary with a *partial* slice of root power instead of full SUID?

> A: That's a Linux **capability**. Find them with `getcap -r / 2>/dev/null` → look up what that specific capability grants (e.g. `cap_setuid` = "this binary can make itself root").

Q: Is there a scheduled job running as root that I can tamper with?

> A: Check `/etc/crontab`, `/etc/cron.d/`, `crontab -l` → if the called script is writable by me, inject a payload and wait for the schedule.

Q: Does some privileged process call a command without specifying its full path?

> A: That's a PATH hijack opportunity. Find writable directories with `find / -writable 2>/dev/null | ...` → if one sits earlier in `$PATH` than the real command's location, drop a malicious file with that exact name there.

Q: Is there a network file share that doesn't properly restrict root access?

> A: Check `/etc/exports` for `no_root_squash` + a writable share. If I already have root *somewhere else*, I can mount the share, drop a SUID payload while root, then trigger it from the low-priv target.

Q: I found a password hash — how do I turn it into a plaintext password?

> A: Feed it to `john` or `hashcat` against a wordlist (`rockyou.txt` is the standard one).

Q: I want a shell sent back to my attacking machine — what's the go-to one-liner?

> A: A bash reverse shell over `/dev/tcp` — exists, look up exact syntax when needed.

WINDOWS

The recurring shape of every service-based technique

Q: In general, how do I know if a Windows service is exploitable?

> A: Ask these four things, in order:
>
> 1. Does it run as SYSTEM? → `sc qc <service>`, check `SERVICE_START_NAME`
> 2. Can I write to its *config*? → `accesschk.exe` against the service name
> 3. Can I write to its *file path* (including ambiguous unquoted paths)? → `accesschk.exe` against the folder/file
> 4. Can I write to its *registry entry*? → `accesschk.exe` against `HKLM\...\Services\<name>` If any answer is yes, redirect it to my payload and restart the service.

Q: A service's binary path has spaces and no quotes — why does that matter?

> A: Windows tries multiple interpretations of the ambiguous path, stopping at the first match — if I can write to one of the earlier folder guesses, my file runs instead of the real one.

Q: The file *itself* (not the config or registry) is writable — what do I do?

> A: Just overwrite it directly, same filename. No config trickery needed.

Q: Is there a program that just launches automatically for anyone who logs in?

> A: Check the registry Run key (`HKLM\...\CurrentVersion\Run`) and the shared StartUp folder — if either the file or the folder is writable, replace/plant a shortcut, then wait for (or trigger) a login.

Q: Is there a system-wide policy letting any user install software with full privileges?

> A: Check `AlwaysInstallElevated` in both `HKCU` and `HKLM` — if both are `1`, build a malicious `.msi` and install it directly.

Q: Is there a scheduled task/script I can tamper with?

> A: Same logic as Linux cron — check the script's write permissions, append a payload line, wait for the trigger.

GUI and token-based tricks — "abusing legitimate features"

Q: An application is running elevated — can its file-open dialog be abused to get a shell?

> A: Yes — Windows common file dialogs can interpret `file://...` paths as commands to execute, inheriting the parent app's privilege level. (This is a known category — check LOLBAS-style resources for which GUI apps are exploitable this way.)

Q: I have a shell as a low-privilege service account — how do I go from there to SYSTEM without any file/registry misconfiguration?

> A: Check `whoami /priv` for `SeImpersonatePrivilege` + `SeAssignPrimaryTokenPrivilege`. If both are present, a "Potato-family" tool (RoguePotato, PrintSpoofer, etc.) can trick a SYSTEM process into authenticating to something I control, letting me steal and reuse its token.

Q: How do these Potato tools actually trick SYSTEM into connecting to me?

> A: They abuse a legitimate Windows feature that expects a "call back here" address (DCOM/OXID resolution, or Print Spooler notification pipes) and substitute their own listener as that address.

Credentials already exposed — "no exploit needed"

Q: Did someone leave a plaintext password somewhere it shouldn't be?

> A: Search the registry for the word "password" (`reg query HKLM /f password ...`), and check the AutoLogon key specifically (`...\winlogon`).

Q: Are there cached/saved logins I can reuse without ever seeing the password?

> A: `cmdkey /list` shows saved credentials → `runas /savecred` lets me run a program as that account silently.

Q: Is there an old backup of the password database lying around?

> A: Look for stray copies of the SAM/SYSTEM files (e.g. `C:\Windows\Repair\`) → dump hashes with a tool like `creddump7` → crack with `hashcat`.

Q: I have a password hash but can't crack it — is there a way to use it directly?

> A: Yes — **pass-the-hash**. Windows checks hashes, not plaintext, so a tool like `pth-winexe` can authenticate using the raw hash alone.

Automation — "tools that run this whole catalog for me"

Q: Is there a tool that checks all of this automatically instead of me doing it by hand?

> A: Yes.
> - Linux → `linpeas.sh`, `LinEnum.sh`

> - Windows → `winPEASany.exe`, `Seatbelt.exe`, `PowerUp.ps1`, `SharpUp.exe` These just automate the same checklist above across every service/file/registry key/process at once.

Q: I found an unusual binary/service — how do I know if it's exploitable without researching it myself?

> A: Look it up on **GTFOBins** (Linux) or **LOLBAS** (Windows) — community-maintained catalogs of exactly which common tools have documented privilege-escalation abuse patterns.

The transferable skill (say this to yourself on any new box)

> "What here has more power than me? What does it depend on? Can I touch that dependency?"

That's the entire card catalog entry. Everything else — `accesschk` flags, `sc qc` syntax, `msfvenom` parameters — is just the shelf number you look up once you know which book you need.

The 8-step worklist — the actual thought process, in order

This is the walk you do, top to bottom, on any unfamiliar box, Windows or Linux. It's the "table of contents" for the whole card catalog above — each step points back at a cluster of Q&As.

1. Who am I, what am I explicitly allowed to do?

> `whoami /priv` (Windows) / `id` + `sudo -l` (Linux) — establishes your starting privileges and any pre-approved elevated commands.

2. Is the OS/kernel old or unpatched?

> Sets up whether a straight-up exploit (kernel, or a known CVE for installed software) is even on the table.

3. What runs automatically, and as whom?

> Services + scheduled tasks (Windows) / cron + systemd timers (Linux) — the list of "things with a schedule or trigger that run with elevated identity."

4. What always runs with someone else's identity baked in?

> SYSTEM-owned services (Windows) / SUID binaries (Linux) — things that execute as a fixed, privileged owner no matter who launches them.

5. What partial/special powers exist?

> Impersonation privileges like `SeImpersonatePrivilege` (Windows) / Linux capabilities like `cap_setuid` (Linux) — a slice of root/SYSTEM power granted without full elevation.

6. Where does the system "guess" or resolve something ambiguously?

> Unquoted service paths (Windows) / PATH order and commands called without a full path (Linux) — anywhere the OS has to interpret something instead of being told exactly where to look.

7. Are credentials sitting somewhere they shouldn't be?

> Registry/cached credentials/SAM dumps (Windows) / config files, history, world-readable secrets (Linux) — no exploit required, just someone else's mistake.

8. Is there a trust boundary that isn't enforced?

> Potato-style token theft (Windows) / NFS `no_root_squash` (Linux) — a feature that works exactly as designed, but trusts something it shouldn't across a boundary (process-to-process, or machine-to-machine).

How to use this list in practice: walk it top to bottom on a new box, checking each one before moving to the next. You won't find something at every step — that's expected. The point isn't "find all eight," it's "know all eight exist, so you never walk away from a box having only checked two or three of the ways in." Tools like linpeas/winPEAS are just this list, automated.